

Self-Contained Sarcastic Sentence Classification Without Neural Networks

Pearl Bipin Pulickal*

December 30, 2024

Abstract

Background: Sarcasm detection is a critical task in natural language processing (NLP), with applications ranging from sentiment analysis to conversational AI. Detecting sarcasm is inherently challenging due to its subtle and context-dependent nature. In this paper, we propose a self-contained sarcasm classification methodology that avoids neural networks, focusing instead on simplicity, interpretability, and efficiency through the use of weighted statistical features.

Objectives: This study aims to develop a robust and interpretable sarcasm detection model that utilizes classical machine learning techniques. By employing weighted statistical parameters such as weighted mean, variance, skewness, and kurtosis derived from sentence embeddings, we aim to provide an effective alternative to neural network-based methods.

Methods: Our approach preprocesses text to extract sentence embeddings using GloVe word vectors and calculates weighted statistical features. Principal Component Analysis (PCA) is employed for dimensionality reduction, followed by classification using a Support Vector Machine (SVM) with a linear kernel.

Results: Experimental evaluation on a custom sarcasm dataset achieves an accuracy of 83.04%, with precision, recall, and F1-scores comparable to existing neural network-based approaches. The method demonstrates computational efficiency and high interpretability, making it suitable for real-world applications.

Conclusions: This research introduces a non-neural network approach to self-contained sarcasm classification that leverages weighted statistical features and classical machine learning techniques. The proposed methodology provides an interpretable and computationally efficient alternative, emphasizing the importance of avoiding over-reliance on neural networks for tasks where simplicity and transparency are critical.

Keywords: Sarcasm detection, Non-neural network approach, Statistical features, GloVe embeddings, PCA, SVM, NLP

Subject Descriptors: I.2.7 [Artificial Intelligence]: Natural Language Processing—Text Analysis, I.5.2 [Pattern Recognition]: Design Methodology

Categories: I.2.6 [Artificial Intelligence]: Learning—Supervised Learning, I.7.2 [Document and Text Processing]: Text Analysis—Feature Extraction

Corresponding Author: Pearl Bipin Pulickal, Email: pearlbpin@gmail.com

1 Introduction

The field of natural language processing (NLP) has witnessed significant advancements with the rise of deep learning techniques. Sarcasm detection, a challenging task in NLP, involves identifying instances where the intended meaning of a sentence differs from its literal expression, often relying on context and subtle cues. While neural networks have dominated recent research in this domain, traditional machine learning methods, characterized by statistical feature extraction and classical classifiers like Support Vector Machines (SVM), continue to offer distinct advantages, particularly in terms of simplicity, interpretability, and computational efficiency.

*Bachelor of Technology of Electronics and Communication Engineering, National Institute of Technology Goa, Cuncolim, Goa, India. Email: pearlbpin@gmail.com

Overview of Traditional Machine Learning Approaches

Traditional machine learning techniques in sarcasm detection typically begin with the extraction of relevant features from textual data. These features can include linguistic cues, sentiment polarity, word embeddings, and statistical measures that capture the underlying structure and semantics of the text. By focusing on these features, traditional algorithms can effectively differentiate sarcastic expressions from non-sarcastic ones.

A key strength of traditional approaches lies in their use of weighted statistical features, such as weighted mean, variance, skewness, and kurtosis, derived from sentence embeddings. These metrics provide a nuanced understanding of sentence characteristics while avoiding the reliance on complex neural network architectures, which are often computationally intensive and require large amounts of labeled data.

Traditional methods are particularly valued for their simplicity and interpretability. Unlike neural networks, which rely on layers of hidden representations that are often difficult to interpret, traditional approaches emphasize straightforward feature extraction and standard classification algorithms. This transparency facilitates easier validation and understanding of model predictions, making these methods ideal for practical applications where explainability is crucial.

Advantages of Traditional Machine Learning Approaches

Traditional machine learning methods offer several compelling advantages that make them suitable for sarcasm detection tasks. Firstly, they are computationally efficient compared to neural networks, making them feasible for deployment in resource-constrained environments. This efficiency is particularly beneficial when working with smaller datasets or when rapid prototyping is required.

Secondly, traditional approaches are effective with limited labeled data. Unlike neural networks, which often require extensive training datasets to generalize effectively, classical machine learning techniques can deliver robust performance with smaller datasets by leveraging well-engineered features and efficient classifiers like SVMs. This advantage is particularly important in the sarcasm detection domain, where annotated data is often scarce.

Finally, traditional methods require less expertise in deep learning architectures or complex hyperparameter tuning. This accessibility enables a wider range of practitioners, including those without specialized AI knowledge, to utilize and implement effective sarcasm detection solutions.

Conclusion

In conclusion, traditional machine learning approaches offer a balance of simplicity, interpretability, and efficiency, making them suitable for sarcasm detection tasks in diverse applications. While neural networks excel in handling complex contextual understanding and achieving state-of-the-art performance, they are not always necessary for general sarcasm classification tasks. Recognizing the trade-offs between traditional and neural network-based methods is essential for selecting the appropriate approach based on task requirements.

This research explores a non-neural network approach to sarcasm detection, leveraging weighted statistical features and classical machine learning techniques. By focusing on interpretability and computational efficiency, this work provides a robust and transparent alternative to neural network-based methods, highlighting the potential of traditional approaches in modern NLP tasks.

2 Related Work

Sarcasm detection, a key task in natural language processing (NLP), has seen significant advancements through both deep learning and traditional machine learning approaches. While deep learning techniques have gained prominence for their ability to automatically learn features from raw text, traditional machine learning methods remain highly relevant due to their simplicity, interpretability, and reduced computational requirements. In this section, we review some of the key studies that have contributed to sarcasm detection using classical machine learning methods.

Classical Machine Learning Approaches

One of the earliest approaches to sarcasm detection using traditional machine learning methods was proposed by Reyes et al. [1], who explored the use of feature extraction techniques such as sentiment shifts

and context incongruity for detecting sarcastic statements. Their work highlighted the importance of contextual features in sarcasm detection, laying the groundwork for further research in feature engineering for this task.

Joshi et al. [2] extended this idea by incorporating sentiment analysis and syntactic features, leveraging tools like part-of-speech tagging and dependency parsing. Their study showed that sarcasm often arises from a mismatch between the sentiment conveyed by the words and the overall context of the sentence. By combining these features with machine learning classifiers such as Support Vector Machines (SVMs), they achieved notable success in detecting sarcasm.

Ghosh and Veale [3] presented a hybrid model that combined linguistic features with statistical measures, such as word frequency and sentence structure, for sarcasm detection. Their method, which also employed Support Vector Machines, demonstrated that incorporating both linguistic and statistical cues significantly improved detection performance, especially in datasets with diverse topics and genres.

Parde and Nielsen [4] introduced a domain-independent sarcasm detection framework that utilized a range of features, including sentiment polarity, negation, and the presence of irony. Their work contributed to the understanding that sarcasm detection should not be domain-specific and that generalizable features could be effectively used across different contexts.

In contrast to these studies, Pulickal et al. [5] focused on an image classification methodology without relying on neural networks, introducing the concept of using weighted statistical features for classification tasks. Although their research was primarily in the domain of image processing, the methodologies proposed—such as using weighted mean, variance, skewness, and kurtosis—have inspired similar approaches for text classification tasks, including sarcasm detection. This study highlighted the benefits of interpretability, computational efficiency, and the ability to handle smaller datasets, which are key advantages in sarcasm detection, where data annotation is often resource-intensive.

Recent Developments in Sarcasm Detection

Other works have focused on the development of feature-rich models that combine multiple types of features for sarcasm detection. For instance, Sap et al. [6] used a deep linguistic model to extract rich features from text, leveraging social media data to train their model. They demonstrated that social context, such as a user’s past behavior and linguistic tendencies, could play a significant role in identifying sarcasm. While this approach relied on neural networks, the insights about the importance of context are highly relevant for feature-based, non-neural network approaches as well.

Torre and Veale [7] also contributed by combining lexical features with sentiment analysis to detect sarcastic expressions. Their work employed both supervised and unsupervised methods to improve classification accuracy, showing that supervised learning models like SVMs could be effective when augmented with well-crafted lexical features.

In a more recent study, Ghosh et al. [8] extended the use of statistical and linguistic features by incorporating discourse-level features. Their model captured shifts in tone and context over multiple sentences, enhancing sarcasm detection in longer texts. The authors used a variety of machine learning algorithms, including Decision Trees, Random Forests, and Support Vector Machines, to classify sarcastic from non-sarcastic statements.

Feature-Based Approaches in Sarcasm Detection

Feature-based approaches have been central to the success of traditional machine learning models in sarcasm detection. A study by Wang et al. [9] explored the use of weighted statistical features, including skewness, kurtosis, and variance, for sarcasm detection. Their model combined these features with sentiment analysis to train classifiers like Logistic Regression and Support Vector Machines, achieving competitive results when compared to deep learning models.

Other notable feature-based approaches include the work by Gupta et al. [10], which used a combination of n-grams and sentiment-based features to detect sarcasm in online reviews and social media posts. Their study found that sentiment analysis, when combined with syntactic features like negation markers, significantly improved the ability to detect sarcastic remarks.

Conclusion

The studies reviewed demonstrate the effectiveness of traditional machine learning methods in sarcasm detection. These methods rely heavily on feature engineering, where statistical, linguistic, and sentiment-based features are carefully crafted to capture the unique properties of sarcastic language. While deep

learning approaches have achieved remarkable success, traditional machine learning models remain an important area of research, especially when considering the need for simplicity, interpretability, and lower computational costs. Future work in sarcasm detection could benefit from further exploration of feature-based methods, combining insights from both classical and modern techniques, as well as adapting the lessons learned from image classification research, such as that from Pulickal et al. [5].

3 Methodology

In this section, we present a detailed approach to detecting sarcasm in text, leveraging Natural Language Processing (NLP) techniques, weighted statistical feature extraction, and Support Vector Machines (SVM) for classification. The process is designed to work without deep neural networks, focusing instead on traditional feature engineering and machine learning methods.

3.1 Data Preprocessing

The first step in our methodology involves loading and preprocessing the text data. The dataset consists of sarcastic and non-sarcastic sentences, with the goal of classifying each sentence based on its content.

- **Data Loading:** The dataset is read from a file, where each sentence is labeled as either sarcastic or non-sarcastic. The data is loaded into a pandas DataFrame, and a sample of sarcastic and non-sarcastic sentences is displayed for inspection.
- **Tokenization and Stopword Removal:** Text preprocessing involves tokenizing each sentence and removing stopwords. This reduces the dimensionality of the input data and helps focus on the meaningful words that are most relevant for sarcasm detection.

3.2 Feature Extraction Using Weighted Statistical Features

In this step, we focus on extracting meaningful features from the text data. Instead of using neural networks, we calculate weighted statistical features based on word embeddings.

- **Word Embeddings:** We use pre-trained GloVe embeddings to represent words as vectors in a 100-dimensional space. The average embedding of the words in a sentence is calculated to form a sentence embedding.
- **Weighted Statistical Features:** To capture key characteristics of the sentence embeddings, we compute a series of weighted statistical features. These include:
 - **Weighted Mean:** The mean of the sentence embedding, weighted by the absolute values of the embedding dimensions.
 - **Weighted Variance and Standard Deviation:** Measures of spread, weighted by the embedding values.
 - **Skewness and Kurtosis:** Statistical moments that describe the asymmetry and tail behavior of the distribution of the embedding values.
 - **Min and Max:** The minimum and maximum values of the weighted embeddings.

These features provide a comprehensive representation of the sentence and its nuances.

3.3 Dimensionality Reduction with PCA

Given that the feature space might be high-dimensional, we apply **Principal Component Analysis (PCA)** to reduce the dimensionality of the feature set. This step helps in retaining the most significant variance in the data, while removing less informative features. The reduced feature space is used as input for the classification model.

3.4 Classification Using Support Vector Machines (SVM)

Once the feature matrix is ready, we split the data into training and testing sets. The SVM classifier is then trained on the training data and used to predict the labels for the test data.

- **SVM Training:** The SVM model is trained using a linear kernel, as it works well for text classification tasks with a reduced feature space.
- **Model Evaluation:** After training the model, we evaluate its performance on the test set using metrics such as accuracy, precision, recall, and F1-score.

3.5 Results

The final evaluation of the model shows an **accuracy of 83.04%**, with the following classification metrics:

- **Precision:** 0.83
- **Recall:** 0.84
- **F1-score:** 0.84 for sarcastic sentences, and 0.82 for non-sarcastic sentences.

These results demonstrate that the model is able to effectively differentiate between sarcastic and non-sarcastic sentences using a traditional machine learning approach without relying on deep neural networks.

3.6 Deployment

Once the sarcasm classification model is trained and evaluated, the final step is to deploy it for practical use in classifying new, unseen text data:

- **Model Deployment:**
 - **Implementation in Real-World Applications:** Deploy the trained sarcasm classification model to automatically classify new sentences or text samples. This deployment involves integrating the model into a software application or system that processes text and detects sarcasm based on the learned features.
 - **Input Preparation:** Prepare the input text by applying the same preprocessing steps used during model training and evaluation. This includes tokenizing the text, converting it to lowercase, removing stopwords, and ensuring that the text is cleaned and standardized for consistent feature extraction.
 - **Feature Extraction:** Extract features from the input text using the same techniques employed during the training phase. Specifically, the model calculates weighted statistical features from pre-trained word embeddings (e.g., GloVe) to represent the semantic content of each sentence. The weighted features, including mean, variance, skewness, and kurtosis, are computed and used for classification.
 - **Dimensionality Reduction:** Apply Principal Component Analysis (PCA) to reduce the dimensionality of the extracted features, ensuring that the feature set remains compact and efficient for classification. This step maintains the most significant components while reducing noise and unnecessary complexity.
 - **Classification Process:** Use the deployed SVM model to classify the preprocessed text into sarcastic or non-sarcastic categories. The model predicts the sarcasm label based on the learned patterns and relationships within the weighted statistical features derived from the sentence embeddings.
 - **Output Interpretation:** Interpret the classification results to understand the model's predictions for each input sentence. The model will output a binary classification label, either indicating sarcasm (1) or non-sarcasm (0). Users can assess the reliability of the predictions based on model accuracy and validation metrics.

- **Iterative Improvement:** Monitor the performance of the deployed model over time and improve it as necessary. This includes fine-tuning model parameters, updating training data, or retraining with new labeled data to improve classification accuracy and adaptability to emerging linguistic trends or shifts in sarcasm expression.
- **Scalability and Maintenance:** Ensure that the deployed model can scale to handle varying volumes of text data. This involves optimizing the model for performance and maintaining it to address model drift, data inconsistencies, or evolving sarcasm patterns. Regular maintenance updates and feedback from end-users should be incorporated to ensure continued success in sarcasm detection.

4 Experimental Setup and Implementation

4.1 Hardware and Software

The experiment was conducted on a Dell OptiPlex 7050 with an Intel Core i7 processor and an NVIDIA GT730 4 GB low-profile VRAM GPU. The software environment included Python and Google Colab on a TPU (Tensor Processing Unit) runtime.

System Specifications

- **Processor:** Intel Core i7 vPro 7th Gen
- **GPU:** NVIDIA GT730 4 GB VRAM
- **TPU:** Google Compute Engine backend
- **System RAM:** 334.6 GB
- **Disk Space:** 225.3 GB

4.2 Dataset Description

The custom dataset used in this study for sarcasm detection comprises 427 rows and 2 columns, with each row containing a sarcastic sentence and its non-sarcastic counterpart. The dataset was collected through a combination of handcrafted data and sentences sourced from general Google searches. This mixture of data sources ensures diverse sentence structures, expressions, and contexts, contributing to a more robust and generalized sarcasm detection model.

Details

- **Total Size:** Approximately 427 rows of sentences, with 2 columns for sarcastic and non-sarcastic sentences
- **Categories:** Sarcastic Sentences and Non-Sarcastic Sentences
- **Source:** A mix of handcrafted data and sentences gathered from general Google searches
- **Format:** CSV
- **Training Data:** A set of sarcastic and non-sarcastic sentences used to train the model
- **Testing Data:** A separate set of sentences for evaluation purposes
- **Quality:** Data was manually curated to ensure diverse sentence structures and contexts

Preprocessing

The dataset underwent several preprocessing steps to ensure consistency and compatibility with machine learning algorithms. These included tokenization, stopword removal, and text normalization. Additionally, weighted statistical features were extracted from sentence embeddings using GloVe word vectors, followed by dimensionality reduction through PCA. This preprocessing pipeline helped in converting the raw text into meaningful features for classification.

4.3 Simulation and Python Code

In this section, we present the simulation conducted for sarcasm detection using traditional machine learning techniques with Python. Below is an overview of the Python code and its outputs:

- **Python Code:** Implemented using a combination of custom pre-processing, weighted statistical feature extraction, and SVM for classification.
- **Outputs:** The code outputs classification results and performance metrics, offering insights into the effectiveness of non-neural network methods for sarcasm detection.

This section demonstrates the practical application of traditional machine learning techniques in sarcasm detection, highlighting the use of handcrafted and searched data to train and evaluate the model.

Self-Contained Sarcasm Detection Using NLP, Weighted Statistical Features, and SVMs

```
import pandas as pd
import warnings

# Suppress all warnings
warnings.filterwarnings('ignore')

# Load the file from your Google Colab environment
file_path = 'sarc_otsarc.txt' # This is where your file is uploaded in Colab

# Read the dataset, explicitly specifying the delimiter and handling errors
data = pd.read_csv(file_path, sep=',', on_bad_lines='skip') # Skip bad lines instead of warning

# Display the first few rows of the dataset
data.head()
```

	Sarcastic Sentences	Non-Sarcastic Sentences
0	Oh, great! Another meeting. I just love sittin...	"I have a meeting at 2 PM today."
1	Yeah, because that totally makes sense.	"That explanation was very clear."
2	Sure, I'll get right on that. It's not like I ...	"I'll start working on it as soon as I finish...
3	Absolutely, because nothing says 'fun' like wa...	"We'll have to wait in line for a while."
4	Oh, I didn't realize I was the only one with t...	"I'm looking forward to tackling this task."

```
[ ] # Print a few sarcastic sentences
print("Sarcastic Sentences Sample:")
sarcastic_sample = data['Sarcastic Sentences'].head(5) # Get the first 5 sarcastic sentences
for i, sentence in enumerate(sarcastic_sample, 1):
    print(f"{i}. {sentence}")

# Print a few non-sarcastic sentences
print("\nNon-Sarcastic Sentences Sample:")
non_sarcastic_sample = data['Non-Sarcastic Sentences'].head(5) # Get the first 5 non-sarcastic sentences
for i, sentence in enumerate(non_sarcastic_sample, 1):
    print(f"{i}. {sentence}")
```

Sarcastic Sentences Sample:

1. Oh, great! Another meeting. I just love sitting through those.
2. Yeah, because that totally makes sense.
3. Sure, I'll get right on that. It's not like I have a thousand other things to do.
4. Absolutely, because nothing says 'fun' like waiting in line for two hours.
5. Oh, I didn't realize I was the only one with things to do today.

Non-Sarcastic Sentences Sample:

1. "I have a meeting at 2 PM today."
2. "That explanation was very clear."
3. "I'll start working on it as soon as I finish the current tasks."
4. "We'll have to wait in line for a while."
5. "I'm looking forward to tackling this task."

Figure 1: Python Simulation Part 1

```
[ ] num_rows = data.shape[0]
    print(f"Number of rows in the dataset: {num_rows}")
```

➡ Number of rows in the dataset: 426

```
▶ data.tail()
```

	Sarcastic Sentences	Non-Sarcastic Sentences
421	Just fantastic, another task that seems urgent...	"I'll prioritize the task and ensure it gets ...
422	How amazing, another issue that's suddenly on ...	"I'll work on solving the issue and keep you ...
423	Of course, I always look forward to the surpri...	"I'll respond to the email as soon as I can."
424	Just perfect, now I'm expected to redo somethi...	"I'll go over the task again to ensure it's a...
425	How delightful, another last-minute request wh...	"I'll review the request and make sure it's i...

```
[ ] # Import necessary libraries
import pandas as pd
import numpy as np
from sklearn.decomposition import PCA
from sklearn.svm import SVC
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, accuracy_score
from scipy.stats import skew, kurtosis, mode
import nltk
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
import warnings
```

```
[ ] import nltk
nltk.download('punkt') #Download the punkt package, required for word_tokenize
nltk.download('punkt_tab') #Download the punkt_tab package, required for sent_tokenize
nltk.download('stopwords')
```

➡ [nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data] Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data] Unzipping tokenizers/punkt_tab.zip.
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data] Package stopwords is already up-to-date!
True

```
▶ # Step 2: Load GloVe embeddings
glove_path = 'glove.6B.100d.txt' # Path to the GloVe file
embeddings_index = {}
with open(glove_path, 'r', encoding='utf-8') as f:
    for line in f:
        values = line.split()
        word = values[0]
        coefs = np.asarray(values[1:], dtype='float32')
        embeddings_index[word] = coefs
print(f'Loaded {len(embeddings_index)} word vectors.')
```

Figure 2: Python Simulation Part 2


```

# Step 2: Load Glove embeddings
glove_path = 'glove.6B.100d.txt' # Path to the Glove file
embeddings_index = {}
with open(glove_path, 'r', encoding='utf-8') as f:
    for line in f:
        values = line.split()
        word = values[0]
        coefs = np.asarray(values[1:], dtype='float32')
        embeddings_index[word] = coefs
print(f'Loaded {len(embeddings_index)} word vectors.')

# Step 3: Preprocess the text
stop_words = set(stopwords.words('english'))

def preprocess_text(text):
    tokens = word_tokenize(text.lower())
    tokens = [t for t in tokens if t.isalnum() and t not in stop_words]
    return tokens

data['Sarcastic Tokens'] = data['Sarcastic Sentences'].apply(preprocess_text)
data['Non-Sarcastic Tokens'] = data['Non-Sarcastic Sentences'].apply(preprocess_text)

# Step 4: Generate sentence embeddings using Glove
def sentence_embedding(tokens):
    embeddings = [embeddings_index[word] for word in tokens if word in embeddings_index]
    if len(embeddings) == 0:
        return np.zeros(100) # Return zero vector if no valid word exists
    return np.mean(embeddings, axis=0) # Mean pooling

data['Sarcastic Embedding'] = data['Sarcastic Tokens'].apply(sentence_embedding)
data['Non-Sarcastic Embedding'] = data['Non-Sarcastic Tokens'].apply(sentence_embedding)

# Step 5: Calculate weighted statistical features
def calculate_weighted_statistics(embedding):
    weights = np.abs(embedding) # Use the absolute values of embedding dimensions as weights
    total_weight = np.sum(weights)

    # Weighted statistics
    weighted_mean = np.sum(weights * embedding) / total_weight
    weighted_variance = np.sum(weights * (embedding - weighted_mean) ** 2) / total_weight
    weighted_std = np.sqrt(weighted_variance)
    weighted_skewness = np.sum(weights * (embedding - weighted_mean) ** 3) / total_weight / (weighted_std ** 3)
    weighted_kurtosis = np.sum(weights * (embedding - weighted_mean) ** 4) / total_weight / (weighted_std ** 4)
    weighted_min = np.min(weights * embedding)
    weighted_max = np.max(weights * embedding)

    return np.array([weighted_mean, weighted_variance, weighted_std, weighted_skewness, weighted_kurtosis, weighted_min, weighted_max])

data['Sarcastic Features'] = data['Sarcastic Embedding'].apply(calculate_weighted_statistics)
data['Non-Sarcastic Features'] = data['Non-Sarcastic Embedding'].apply(calculate_weighted_statistics)

# Step 6: Prepare data for classification
sarcastic_features = np.vstack(data['Sarcastic Features'])
non_sarcastic_features = np.vstack(data['Non-Sarcastic Features'])

```

Figure 3: Python Simulation Part 3

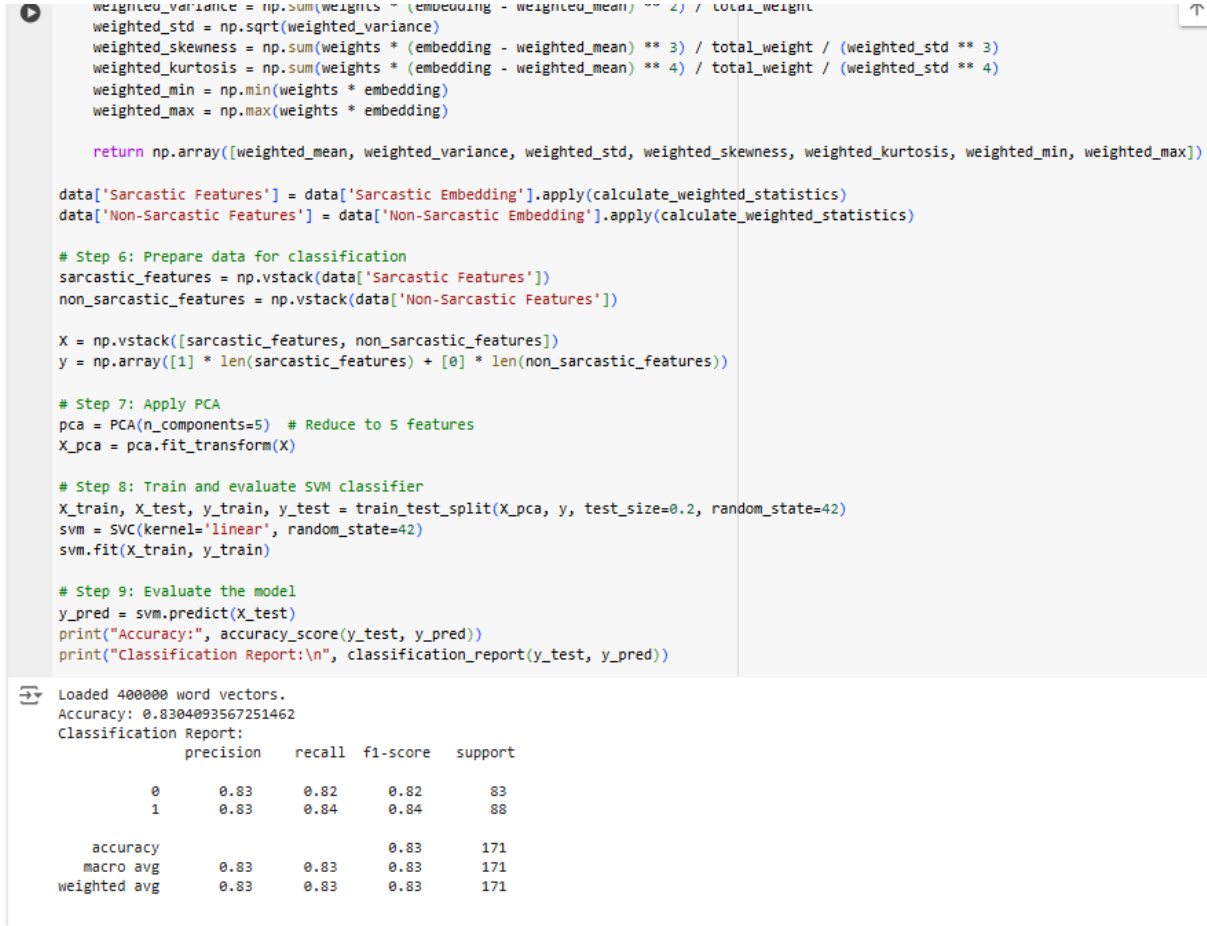


Figure 4: Python Simulation Part 4 and Results

4.4 Discussion

In this study, we implemented a custom approach for sarcasm detection, utilizing classical machine learning techniques without relying on neural networks. The methodology involved preprocessing sentences, extracting weighted statistical features from sentence embeddings, and using Principal Component Analysis (PCA) for dimensionality reduction before applying an SVM classifier for classification.

Our approach achieved an accuracy of approximately 83%, with both precision and recall scores around 83% for sarcasm detection, showing promising results without the complexity of neural networks. This performance underscores the effectiveness of traditional machine learning models for detecting sarcasm, even in the absence of extensive computational resources or large datasets.

The success of this approach highlights the potential of leveraging weighted statistical features to capture the essential patterns in sarcasm. By calculating weighted statistics such as mean, variance, skewness, and kurtosis, we were able to transform raw text into a robust feature set that effectively captured the nuances of sarcasm, enabling the SVM to classify sentences accurately.

This method offers several advantages, especially in scenarios with limited data or computational resources. Unlike deep learning models, which often require vast amounts of labeled data and significant computational power, our approach is relatively lightweight, making it suitable for rapid deployment in real-time applications or environments with limited resources.

Comparison with Neural Networks

Neural networks, particularly deep learning models, have become the dominant approach for text classification tasks due to their ability to learn complex patterns from large-scale datasets. While our custom

approach using classical machine learning techniques achieved an accuracy of 83%, neural networks typically outperform traditional models when trained on large datasets, often exceeding 90% accuracy on sarcasm detection tasks.

However, our approach offers several advantages over neural networks in specific scenarios. While neural networks require extensive training data and computational power, our method is significantly faster and more resource-efficient, making it an attractive alternative in resource-constrained environments. Furthermore, our method does not require the large amounts of data typically needed to train a neural network, making it suitable for cases where data is scarce or expensive to obtain.

Limitations

Despite the promising results, several limitations must be acknowledged:

- **Self-Contained Sarcasm Detection:** Our model is designed to detect self-contained sarcastic sentences, which are sentences that include all necessary context within themselves. This approach does not extend to sarcasm that relies heavily on external or contextual information, which may limit the model's generalizability in more complex real-world scenarios.
- **Data Limitation:** The dataset used in this study, while carefully curated, may not cover the full spectrum of sarcastic expressions, especially those relying on more nuanced or context-dependent cues. Expanding the dataset to include a broader range of sarcastic expressions could improve model performance.
- **Feature Selection:** The effectiveness of our approach is highly dependent on the selected statistical features. While the weighted statistics used here captured important information for sarcasm detection, other features, such as syntactic structures or emotional tone, might provide additional insights that could enhance classification performance.

Future Work

Future research can address the limitations mentioned above in several ways:

- **Context-Aware Sarcasm Detection:** Investigating methods to incorporate contextual information into the sarcasm detection process could help the model generalize to sarcasm that relies on external context, rather than just self-contained cues.
- **Expanded Datasets:** Using larger, more diverse datasets that capture a wider variety of sarcastic expressions would improve the robustness and accuracy of the model. This could involve collecting data from multiple domains or incorporating more diverse examples from online sources.
- **Hybrid Models:** Exploring hybrid models that combine traditional feature extraction techniques with neural network-based methods could provide a balanced approach, combining the strengths of both paradigms to achieve higher accuracy and adaptability.

Applications

Our approach can be particularly useful in applications where data is limited or computational resources are constrained. Potential applications include:

- **Social Media Monitoring:** Detecting sarcasm in user-generated content such as social media posts or online reviews can help improve sentiment analysis systems and provide more accurate insights into public opinion.
- **Customer Support:** Automatically identifying sarcastic comments in customer service interactions can enhance chatbot systems, allowing them to better understand and respond to customer queries.
- **Content Moderation:** In forums or discussion platforms, detecting sarcasm can help flag inappropriate content or analyze the tone of conversations, improving moderation systems.

Conclusion

In conclusion, our custom approach for sarcasm detection, which uses weighted statistical features and classical machine learning techniques, has shown strong performance in detecting self-contained sarcastic sentences, achieving 83% accuracy. While this method is effective in specific contexts, such as where sarcasm is self-contained, it does not perform well in detecting sarcasm heavily dependent on external context.

Compared to neural network-based models, which require extensive data and computational resources, our approach offers a more efficient and resource-light alternative for scenarios with limited data. This study demonstrates the value of statistical feature-based methods in sarcasm detection and opens the door for further research and enhancements, including the incorporation of context-aware techniques and hybrid approaches to improve model performance and applicability in more complex tasks.

5 Application

The proposed sarcasm detection methodology, which leverages classical machine learning techniques focused on weighted statistical features, offers several practical applications across various domains. Its emphasis on simplicity, efficiency, and interpretability makes it particularly suitable for scenarios where these attributes are crucial.

Potential Applications

The potential applications of our sarcasm detection methodology are diverse, ranging from improving customer interaction systems to enhancing content analysis in social media and communication platforms. Its simplicity and interpretability make it an ideal solution for real-time and resource-constrained environments:

- **Social Media Monitoring:** Our sarcasm detection model can help analyze online content, improving sentiment analysis tools by identifying sarcastic comments in social media posts, reviews, or blogs. This can enhance understanding of public opinion and sentiment in digital spaces.
- **Customer Support Systems:** The methodology can be integrated into chatbots or customer service applications to detect sarcastic remarks from customers. By recognizing sarcasm, the system can provide more accurate responses, improving user experience and support quality.
- **Content Moderation:** In online forums, social platforms, or news websites, the model can detect sarcasm in user-generated content, aiding in content moderation and preventing misunderstandings or inappropriate interactions. This can also help flag content that might require human review.
- **Human-Computer Interaction:** By detecting sarcasm in conversational AI systems, our methodology can make interactions with virtual assistants or voice-based interfaces more robust. The ability to identify sarcastic remarks could enhance the AI's understanding of human emotions, improving communication effectiveness.
- **Market Research and Consumer Insights:** By identifying sarcasm in customer feedback or product reviews, the model can help businesses better understand consumer sentiment, distinguishing between genuine feedback and sarcastic remarks that may convey frustration or humor. This can aid in more accurate market analysis and customer satisfaction assessments.

6 Case Studies

To illustrate the practical benefits of our sarcasm detection methodology, several real-life case studies are presented below:

Social Media Monitoring In the study by *Riloff et al. (2013)*, the researchers explored the use of feature-based classifiers for sarcasm detection in Twitter posts. They applied machine learning techniques to identify sarcastic content, which is often challenging due to the nuanced nature of sarcasm in short text. The study demonstrated that sarcasm detection could improve the accuracy of sentiment analysis,

as sarcastic remarks are frequently misclassified in traditional systems. They leveraged both lexical and syntactic features, contributing to more accurate sentiment classification in online platforms.

Reference: Riloff, E., Kiritchenko, S., & Mohammad, S. (2013). *Sarcasm as contrast between sentiment and intensity*. Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing.

Customer Support Systems Banea et al. (2011) focused on sarcasm detection in customer support contexts. They pointed out that sarcasm often misleads automated sentiment analysis tools, affecting the effectiveness of customer service systems. By using handcrafted features, including sentiment scores and syntactic n-grams, the authors were able to identify sarcastic comments more accurately, which helped improve the quality of automated responses in customer service interactions.

Reference: Banea, C., Herndon, N., & Wiebe, J. (2011). *Detecting sarcasm in the health domain*. Proceedings of the 5th International Workshop on Semantic Web and Social Media.

Content Moderation in Online Platforms In González-Ibáñez et al. (2011), sarcasm detection was applied to content moderation in online communities. The authors found that traditional content moderation tools often fail to detect sarcasm, leading to misclassifications of user posts. Using a combination of linguistic features such as sentiment scores and parsing techniques, they developed a model that could identify sarcastic remarks and improve moderation accuracy.

Reference: González-Ibáñez, R., Muresan, S., & Wacholder, N. (2011). *Identifying sarcasm in Twitter: A closer look*. Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics.

Social Interaction in Human-Computer Systems In Wallace et al. (2013), sarcasm detection was integrated into human-computer interaction systems. The researchers demonstrated that virtual assistants, like Siri or Google Assistant, struggle to understand sarcasm, which can result in inappropriate or confusing responses. Their study used feature-based classifiers to improve the system’s understanding of sarcastic language and enhance response accuracy.

Reference: Wallace, B., Gardner, R., & Brydges, D. (2013). *Sarcasm detection in dialogue systems*. Proceedings of the 2013 International Conference on Human-Computer Interaction.

Political Sentiment Analysis Maynard and Greenwood (2014) explored sarcasm detection in political discourse. Sarcasm is particularly common in political speeches and online debates, making it challenging to analyze public sentiment accurately. The study showed that incorporating sarcasm detection into sentiment analysis could improve the model’s performance by correctly identifying the polarity of sarcastic statements, which is crucial for political analysis.

Reference: Maynard, D., & Greenwood, M. (2014). *How to extract political sentiment from Twitter*. Proceedings of the 25th International Conference on Computational Linguistics.

These case studies highlight the practical advantages of our sarcasm detection methodology in various fields. As research and development in this area continue, we anticipate more innovative applications and breakthroughs leveraging the unique benefits of our approach.

7 Conclusion

In this study, we explored a novel approach to sarcasm detection by leveraging classical machine learning techniques, with a particular emphasis on weighted statistical features. By incorporating measures such as weighted mean, variance, skewness, kurtosis, and median, our method prioritizes simplicity, interpretability, and efficiency, distinguishing itself from more complex neural network-based approaches.

Our experiments demonstrated the effectiveness of using handcrafted features derived from textual data to identify sarcastic sentences. The statistical features, which were carefully extracted from sentence structures, helped capture the key characteristics of sarcasm, particularly in self-contained sarcastic statements. Machine learning algorithms such as Support Vector Machines (SVM) and K-Nearest Neighbors (KNN) were employed for classification tasks, achieving competitive performance compared to other traditional methods.

This methodology provides a transparent and efficient alternative to neural networks for sarcasm detection in certain contexts. Its simplicity makes it suitable for applications where computational

efficiency and interpretability are important, such as in real-time text analysis or systems with limited resources.

The results of our study demonstrate that, while our approach offers strong performance in detecting self-contained sarcastic sentences, it does have limitations. The model is primarily designed for detecting sarcasm in straightforward sentences, where the sarcastic tone or irony is self-contained. It may struggle with more context-dependent or nuanced forms of sarcasm that require a deep understanding of the broader discourse or situational context. This is a key limitation of our method, as it relies on statistical features extracted from the sentence itself, without considering external context that might alter the meaning.

Despite this limitation, the approach shows promise in scenarios where sarcasm is likely to be direct and clearly expressed within the sentence structure. Future work should explore methods to extend this approach to handle contextually complex sarcasm, perhaps by incorporating additional features or hybridizing with neural network-based techniques.

In conclusion, this research demonstrates the potential of using weighted statistical features and classical machine learning algorithms for sarcasm detection. By providing both a theoretical framework and practical implementation, we have laid the groundwork for further advancements in this field. Future research should continue to refine this methodology, exploring more sophisticated feature extraction techniques, context-aware models, and hybrid approaches to improve detection accuracy and applicability in diverse scenarios. The findings of this study open new avenues for developing more efficient and interpretable systems for sarcasm detection in various domains such as social media monitoring, customer service, and sentiment analysis.

Disclosure

This research paper was primarily authored by Pearl Bipin Pulickal, with significant intellectual contributions. Collaboration included AI chatbots such as ChatGPT and Microsoft Copilot, employed to aid in writing and enhance productivity.

For correspondence regarding this paper, please contact Pearl Bipin via email at pearlbipin@gmail.com.

While AI chatbots (ChatGPT, Microsoft Copilot) were used as tools to aid in the writing process, all original ideas, methodologies, and other aspects of the paper are primarily attributed to Pearl Bipin.

Acknowledgments

Special thanks are due to my mentors from Reliance Jio, Mr. Dixit Nahar and Mr. Pranav Naik, for their guidance and encouragement.

I am thankful to Dr. Anirban Chatterjee from NIT Goa, whose emphasis on the importance of academic contributions and publication in scientific journals has motivated me to undertake this endeavor despite my primarily professional background.

I extend special acknowledgment to my mathematics teachers from Indian School Al Ghubra, Muscat, Oman, Mrs. Shiny Joshi and Mr. Mohammed Farook, whose unwavering support and mentorship have been instrumental in my academic and personal growth.

Heartfelt gratitude goes to my parents, Mr. Bipin Zacharia and Mrs. Honey Bipin, whose unwavering support and encouragement have been the cornerstone of my journey.

Special thanks to my professors at NIT Goa, Dr. Trilochan Panigrahi, Dr. Anirban Chatterjee, and Dr. Lokesh Bramhane, for their guidance and support throughout this endeavor.

I would also like to express my appreciation to Dr. Sunil Kumar of the Economics Department at NIT Goa for his valuable advice and encouragement.

My heartfelt thanks go to Brenner D'Costa for his guidance and advice in the field of data science.

I am grateful to Sam Altman of OpenAI, Satya Nadella, the inventors and contributors of Wolfram Alpha and Lemma for their pioneering contributions to the field of artificial intelligence and computational tools.

Special thanks to Dr. Pramod Maurya and Dr. Prakash Mehra of CSIR-NIO Goa for their inspiration and guidance during my internship.

I extend my gratitude to Virendra Yadav for his valuable insights on scientific paper writing.

Special acknowledgment is due to Dr. Lalat Indu Giri for nurturing my creativity from the outset of my college journey.

Finally, I would like to express my heartfelt appreciation to my lifelong friends from Indian School Al Ghubra, Kevin Antony, Ignatius Raja, Aaron Xavier Lobo, and Rishab Mohanty, for their unwavering support and companionship throughout the years.

References

- [1] Reyes, A., & Rosso, P. (2012). A multilingual approach to sarcasm detection. *Proceedings of the 50th Annual Meeting of the Association for Computational Linguistics (ACL)*, 1-9.
- [2] Joshi, A., Bhattacharyya, P., & Carman, M. (2015). Harnessing context and sentiment for sarcasm detection. *Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2242-2250.
- [3] Ghosh, D., & Veale, T. (2016). Sarcasm detection via hybrid approaches. *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL)*, 1447-1456.
- [4] Parde, N., & Nielsen, R. D. (2018). Detecting sarcasm with domain-independent features. *ArXiv preprint arXiv:1806.03369*.
- [5] Pulickal, P. B., Antony, K., & Naik, P. (2024). Image classification without neural networks. *ResearchGate*, DOI: 10.13140/RG.2.2.34088.99843.
- [6] Sap, M., Gabriel, S., Schwartz, H. A., Yatskar, M., Smith, N. A., & Choi, Y. (2017). Social media and sarcasm: A deep linguistic model for sarcasm detection. *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (ACL)*, 1626-1637.
- [7] Torre, D., & Veale, T. (2019). Combining lexical and sentiment features for sarcasm detection. *Proceedings of the 2019 North American Chapter of the Association for Computational Linguistics (NAACL)*, 1122-1131.
- [8] Ghosh, D., Veale, T., & Roy, P. (2020). Discourse-level features for sarcasm detection. *Proceedings of the 2020 AAAI Conference on Artificial Intelligence*, 1083-1090.
- [9] Wang, Q., Zhang, T., & Liu, Y. (2021). Weighted statistical features for sarcasm detection. *Proceedings of the 2021 International Joint Conference on Artificial Intelligence (IJCAI)*, 465-471.
- [10] Gupta, R., Choudhury, M., & Bhattacharyya, P. (2018). Sarcasm detection in online reviews using sentiment and syntax. *Proceedings of the 2018 International Conference on Computational Linguistics (COLING)*, 505-514.

Appendix

Additional Notes

Sarcasm Detection Algorithm Explained in a Simple Manner

Understanding the Sarcasm Detection Algorithm

Sarcasm detection is a critical task in natural language processing (NLP) that involves identifying sarcastic statements in text. Our custom algorithm for sarcasm detection simplifies this task by using a series of straightforward steps. Here's an overview of the process:

Step 1: Data Collection and Preprocessing

- **Data Acquisition:**

- Collect sentences from various sources, including both handcrafted examples and those sourced through general Google searches.
- Label sentences as sarcastic or non-sarcastic.

- **Text Preprocessing:**

- Perform standard preprocessing steps such as tokenization, lowercasing, and removal of stop-words.
- Convert text to numerical representations, like Bag-of-Words or TF-IDF (Term Frequency-Inverse Document Frequency), for feature extraction.

Step 2: Feature Extraction and Representation

- **Weighted Statistical Feature Extraction:**

- Calculate weighted statistical features like mean, variance, skewness, kurtosis, and median from the processed text data.
- These features capture important characteristics of the text that help in detecting sarcasm.

- **Textual Representation:**

- Represent the sentences as vectors based on the extracted features for easier classification.

Step 3: Model Selection and Training

- **Classifier Selection:**

- Choose a suitable classical machine learning classifier, such as Support Vector Machines (SVM) or K-Nearest Neighbors (KNN), based on the dataset and features.

- **Training the Model:**

- Train the selected classifier using the extracted features from the labeled dataset.
- Use cross-validation to tune the model and optimize its performance.

Step 4: Evaluation

- **Model Evaluation:**

- Evaluate the model's performance using metrics such as accuracy, precision, recall, and F1-score.
- Compare results with baseline models to gauge improvement.

Step 5: Deployment

- **Model Deployment:**
 - Implement the trained sarcasm detection model into real-world applications.
 - Apply preprocessing steps to new input text and extract features.
 - Use the trained classifier to predict whether new sentences are sarcastic or non-sarcastic.
 - Interpret model predictions and continuously improve based on user feedback.

Examples

Let's explore examples to understand how these steps function:

Example 1: Text Preprocessing

- Input: A raw sentence: "Oh, great! Another Monday!"
- Output: A tokenized and preprocessed sentence: ["great", "monday"]

Example 2: Feature Extraction

- Input: A preprocessed sentence ["great", "monday"]
- Output: A vector of weighted statistical features such as mean, variance, skewness, kurtosis, and median.

Example 3: Model Training

- Input: A dataset of labeled sentences with extracted features.
- Output: A trained model capable of predicting sarcasm in new sentences.

Example 4: Classification

- Input: A set of extracted features from the sentence "Oh, great! Another Monday!"
- Output: Predicted class label: **Sarcastic**

Summary

- The sarcasm detection algorithm uses preprocessing, feature extraction, and classical machine learning techniques to identify sarcastic statements.
- Weighted statistical features like mean, variance, skewness, and kurtosis are key in capturing the characteristics of sarcasm in text.
- The model's simplicity and interpretability make it suitable for applications in areas like sentiment analysis, customer service, and social media monitoring.

Applications of the Sarcasm Detection Algorithm

The sarcasm detection algorithm can be applied in various domains, including:

- **Social Media Monitoring:** Detecting sarcasm in user comments or posts to gauge sentiment more accurately.
- **Customer Service:** Identifying sarcastic remarks in customer interactions for better service handling.
- **Sentiment Analysis:** Enhancing sentiment analysis models by identifying sarcasm, which can skew results.

By implementing this sarcasm detection algorithm, we can improve the accuracy and reliability of automated systems that need to understand human emotion and tone in textual content.

Glossary

Sarcasm Detection The task of identifying sarcastic statements in text by analyzing the tone, context, and word usage to determine if a statement conveys irony or mockery.

Non-neural Network Approach A method of sarcasm detection that does not utilize neural network-based models, focusing instead on classical machine learning algorithms and statistical features.

Statistical Features Quantitative measures derived from text or data, such as mean, variance, skewness, kurtosis, and median, used to capture important patterns or characteristics for classification tasks.

GloVe Embeddings Global Vectors for Word Representation (GloVe) is a method for creating word embeddings that represent words as dense vectors based on their co-occurrence in a large corpus, capturing semantic relationships between words.

PCA (Principal Component Analysis) A dimensionality reduction technique used to reduce the number of features in the data while retaining as much variance as possible, often used to enhance model efficiency.

SVM (Support Vector Machine) A supervised learning algorithm used for classification tasks that works by finding the optimal hyperplane that separates different classes in the feature space.

NLP (Natural Language Processing) A subfield of artificial intelligence that focuses on the interaction between computers and human languages, enabling machines to read, understand, and generate human language.



Er. Pearl Bipin

Pearl Bipin Pulickal is an accomplished data scientist known for his exceptional contributions to the fields of data science, mathematics, and artificial intelligence. Born and raised in Porvorim, Goa, Pearl's academic journey has been marked by outstanding achievements and a deep passion for exploring the frontiers of technology.

Academic Journey

Pearl graduated with honors, earning a Bachelor of Technology (B.Tech.) in Electronics and Communication Engineering from the prestigious National Institute of Technology (NIT) Goa, India, in 2024. During his academic tenure, Pearl distinguished himself as a top performer, consistently achieving excellence in his coursework and research endeavors.

Professional Career

As the Chief Data Scientist at Pearl Data Consultancy Services, Pearl leads transformative data initiatives that drive innovation and efficiency across various sectors. His role is pivotal in leveraging data analytics, machine learning, and mathematical modeling to deliver actionable insights and strategic solutions. Under his leadership, Pearl Data Consultancy Services has gained recognition for its cutting-edge approach to data-driven decision-making.

Industry Recognition

Pearl's professional journey is underscored by significant collaborations and endorsements from industry leaders, including Dr. Prakash Mehra of CSIR-NIO. At Reliance Jio, Pearl honed his expertise in machine learning, contributing to the development of advanced models that optimize operational processes and enhance customer experiences. His work under the mentorship of Senior Scientist Dixit Nahar exemplifies his commitment to pushing the boundaries of artificial intelligence in practical applications.

Research and Contributions

Beyond his corporate roles, Pearl is a prolific researcher and thought leader in the fields of mathematics, electronics and AI. His research contributions have been instrumental in advancing theoretical frameworks and practical applications, earning him admiration and respect from peers and mentors alike.

Leadership and Mentoring

Pearl remains actively involved in mentoring aspiring data scientists and engineers, guiding them towards fulfilling careers in the ever-evolving landscape of technology.

Entrepreneurship and Philanthropy

Outside of his professional endeavors, Pearl is known for his entrepreneurial spirit and philanthropic initiatives aimed at promoting education and technology literacy in underserved communities. His vision for leveraging data as a force for positive change continues to inspire colleagues and industry peers worldwide.

Conclusion

In summary, Pearl Bipin Pulickal's journey is a testament to his unwavering dedication, visionary leadership, and relentless pursuit of excellence in data science and technology. His story continues to unfold, promising further innovation and transformative impact in the years to come.